

TOPIC 1: INTRODUCTION

Shivani S - 192471030

#1 First Palindromic String

Program

```
def first_palindrome(words):
    for word in words:
        if word == word[::-1]:
            return word
    return ""

words = ["abc", "car", "ada", "racecar", "cool"]
print("First Palindromic String:", first_palindrome(words))
```

Output

First Palindromic String: ada

#2 Find Intersection Values

Program

```
def find_intersection_values(nums1, nums2):
    set1 = set(nums1)
    set2 = set(nums2)

    answer1 = sum(1 for x in nums1 if x in set2)
    answer2 = sum(1 for x in nums2 if x in set1)

    return [answer1, answer2]

nums1 = [2, 3, 2]
nums2 = [1, 2]

print(find_intersection_values(nums1, nums2))
```

Output

[2, 1]

#3 Sum of Distinct Counts

Program

```
def sum_counts(nums):  
    n = len(nums)  
    total = 0  
  
    for i in range(n):  
        distinct = set()  
  
        for j in range(i, n):  
            distinct.add(nums[j])  
            total += len(distinct) ** 2  
  
    return total  
  
nums = [1, 2, 1]  
  
print("Sum =", sum_counts(nums))
```

Output

Sum = 15

#4 Count Equal and Divisible Pairs

Program

```
def count_pairs(nums, k):  
  
    count = 0  
    n = len(nums)  
  
    for i in range(n):  
  
        for j in range(i + 1, n):
```

```
        if nums[i] == nums[j] and (i * j) % k == 0:

            count += 1

    return count

nums = [3,1,2,2,2,1,3]

k = 2

print("Number of pairs:", count_pairs(nums, k))
```

Output

Number of pairs: 4

#5 Find Maximum

Program

```
def find_max(arr):

    if len(arr) == 0:
        return None

    maximum = arr[0]

    for num in arr:

        if num > maximum:

            maximum = num

    return maximum

print(find_max([1,2,3,4,5]))
```

```
print(find_max([7,7,7,7,7]))
print(find_max([-10,2,3,-4,5]))
```

Output

```
5
7
5
```

#6 Find Maximum After Sorting

Program

```
def find_max_after_sort(arr):

    if len(arr) == 0:
        return "List is empty"

    arr.sort()

    return arr[-1]

print(find_max_after_sort([]))
print(find_max_after_sort([5]))
print(find_max_after_sort([3,3,3,3,3]))
```

Output

```
List is empty
5
3
```

#7 Unique Elements

Program

```
def unique_elements(nums):

    unique = []

    for num in nums:
```

```
    if num not in unique:

        unique.append(num)

    return unique

print(unique_elements([3,7,3,5,2,5,9,2]))
print(unique_elements([-1,2,-1,3,2,-2]))
print(unique_elements([1000000,999999,1000000]))
```

Output

```
[3, 7, 5, 2, 9]
[-1, 2, 3, -2]
[1000000, 999999]
```

#8 Bubble Sort

Program

```
def bubble_sort(arr):

    n = len(arr)

    for i in range(n):

        for j in range(0, n-i-1):

            if arr[j] > arr[j+1]:

                arr[j], arr[j+1] = arr[j+1], arr[j]

    return arr

arr = [64,34,25,12,22,11,90]
```

```
print("Sorted Array:", bubble_sort(arr))
```

Output

Sorted Array: [11, 12, 22, 25, 34, 64, 90]

#9 Binary Search

Program

```
def binary_search(arr, key):
```

```
    arr.sort()
```

```
    low = 0
```

```
    high = len(arr)-1
```

```
    while low <= high:
```

```
        mid = (low+high)//2
```

```
        if arr[mid] == key:
```

```
            return mid+1
```

```
        elif arr[mid] < key:
```

```
            low = mid+1
```

```
        else:
```

```
            high = mid-1
```

```
    return -1
```

```
arr = [3,4,6,-9,10,8,9,30]
```

```
key = 10

result = binary_search(arr,key)

if result != -1:

    print("Element found at position", result)

else:

    print("Element not found")
```

Output

Element found at position 7

#10 Merge Sort

Program

```
def merge_sort(arr):

    if len(arr) <= 1:

        return arr

    mid = len(arr)//2

    left = merge_sort(arr[:mid])

    right = merge_sort(arr[mid:])

    result = []

    i = j = 0

    while i < len(left) and j < len(right):

        if left[i] < right[j]:
```

```

        result.append(left[i])

        i += 1

    else:

        result.append(right[j])

        j += 1

    result.extend(left[i:])

    result.extend(right[j:])

    return result

```

```
nums = [5,2,3,1]
```

```
print(merge_sort(nums))
```

Output

```
[1, 2, 3, 5]
```

#11 Out of Boundary Paths

Program

```
def find_paths(m,n,N,row,col):
```

```
    dp=[[0]*n for _ in range(m)]
```

```
    dp[row][col]=1
```

```
    count=0
```

```
    for _ in range(N):
```



```

temp=[[0]*n for _ in range(m)]

for i in range(m):

    for j in range(n):

        if dp[i][j]:

            for x,y in [
                (1,0),
                (-1,0),
                (0,1),
                (0,-1)
            ]:

                ni=i+x
                nj=j+y

                if 0<=ni<m and 0<=nj<n:

                    temp[ni][nj]+=dp[i][j]

            else:

                count+=dp[i][j]

        dp=temp

return count

print(find_paths(2,2,2,0,0))

```

Output

6

#12 House Robber II

Program

```
def rob_linear(nums):

    prev=curr=0

    for num in nums:

        prev,curr=curr,max(
            curr,
            prev+num
        )

    return curr


def rob(nums):

    if len(nums)==1:

        return nums[0]

    return max(
        rob_linear(nums[:-1]),
        rob_linear(nums[1:])
    )


print(rob([2,3,2]))

print(rob([1,2,3,1]))
```

Output

3
4

#13 Climbing Stairs

Program

```
def climb_stairs(n):  
  
    if n <= 2:  
  
        return n  
  
    a = 1  
    b = 2  
  
    for i in range(3, n+1):  
  
        a, b = b, a+b  
  
    return b
```

```
print(climb_stairs(4))
```

```
print(climb_stairs(3))
```

Output

5

3

#14 Unique Paths

Program

```
def unique_paths(m, n):  
  
    dp = [[1]*n for _ in range(m)]  
  
    for i in range(1, m):  
  
        for j in range(1, n):  
  
            dp[i][j] = (
```

```

        dp[i-1][j]
        + dp[i][j-1]
    )

return dp[m-1][n-1]

print(unique_paths(7,3))

print(unique_paths(3,2))

```

Output

```

28
3

```

#15 Positions of Large Groups

Program

```

def large_group_positions(s):

    result = []

    start = 0

    for i in range(1, len(s)+1):

        if i == len(s) or s[i] != s[start]:

            if i-start >= 3:

                result.append(
                    [start, i-1]
                )

            start = i

    return result

```

```
print(
large_group_positions(
"abbxxxxzzy"
)
)
```

```
print(
large_group_positions(
"abc"
)
)
```

Output

```
[[3, 6]]
[]
```

#16 Game of Life

Program

```
def game_of_life(board):
```

```
    rows = len(board)
```

```
    cols = len(board[0])
```

```
    direction = [
(-1,-1),(-1,0),(-1,1),
(0,-1),(0,1),
(1,-1),(1,0),(1,1)
]
```

```
    new = [
[0]*cols
for _ in range(rows)
]
```

```

for i in range(rows):

    for j in range(cols):

        live = 0

        for dx, dy in direction:

            ni = i + dx
            nj = j + dy

            if (
0 <= ni < rows
and
0 <= nj < cols
):

                live += board[ni][nj]

            if board[i][j] == 1:

                if live == 2 or live == 3:

                    new[i][j] = 1

            else:

                if live == 3:

                    new[i][j] = 1

        return new

board = [
[0,1,0],
[0,0,1],

```

```
[1,1,1],  
[0,0,0]  
]
```

```
print(  
game_of_life(board)  
)
```

Output

```
[[0, 0, 0],  
 [1, 0, 1],  
 [0, 1, 1],  
 [0, 1, 0]]
```

#17 Champagne Tower

Program

```
def champagne_tower(  
    poured,  
    query_row,  
    query_glass  
):
```

```
    tower = [  
        [0.0]*(r+1)  
        for r in range(101)  
    ]
```

```
    tower[0][0] = poured
```

```
    for row in range(  
        query_row+1  
    ):
```

```
        for glass in range(  
            row+1  
        ):
```

```
        overflow = (  
tower[row][glass]-1  
) / 2
```

```
        if overflow > 0:
```

```
            tower[  
row+1  
][glass] += overflow
```

```
            tower[  
row+1  
][glass+1] += overflow
```

```
        return min(  
1,  
tower[  
query_row  
][query_glass]  
)
```

```
print(  
champagne_tower(  
1,  
1,  
1  
)  
)
```

```
print(  
champagne_tower(  
2,  
1,  
1  
)  
)
```


Output

0.0

0.5